

AS61B – Fundamentos de Programação

Registros (Estruturas)

13/05/2024

Contextualização

- **Cenário:** Faça um algoritmo que represente um aluno. O usuário deve digitar o RA, nome e endereço do aluno. Em seguida, o algoritmo deve apresentar os dados do aluno.

```
#include <stdio.h>

int main(){
    int ra;
    char nome[50];
    char endereco[50];
    ...
    return 0;
}
```

Contextualização

- Neste contexto, é possível ter inúmeras variáveis para representar esse aluno.
- E se quiséssemos representar 3 alunos? Como ficaria?

```
#include <stdio.h>
```

```
int main(){
    int ra1, ra2, ra3;
    char nome1[50], nome2[50], nome3[50];
    char endereco1[50], endereco2[50], endereco3[50];
    ...
    return 0;
}
```

Contextualização

- Neste contexto, é possível ter inúmeras variáveis para representar esse aluno.
- E se quiséssemos representar 100 alunos? Como ficaria?

Contextualização

- Poderíamos buscar uma solução utilizando variáveis compostas homogêneas:

```
int main(){
    int ra[100];
    char nome[100][50];
    char endereco[100][50];

    //Preenchendo os dados
    ra[0] = 1;
    strcpy(nome[0], "João Carlos");
    strcpy(endereco[0], "Rua um");

    ra[1] = 2;
    strcpy(nome[1], "Pedro da Silva");
    strcpy(endereco[1], "Rua dois");

    ...

    return 0;
}
```

Contextualização

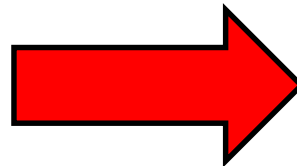
- Esta é uma boa solução?
- Devemos pensar o que estes atributos, RA, nome e endereço, têm em comum.
- Todos eles estão relacionados, pois representam o aluno → abstração.
- Solução: registros (estruturas)

Contextualização

- Cenário:** Faça um algoritmo que represente um aluno. O usuário deve digitar o RA, nome e endereço do aluno. Em seguida, o algoritmo deve apresentar os dados do aluno.

```
#include <stdio.h>

int main(){
    int ra;
    char nome[50];
    char endereco[50];
    ...
    return 0;
}
```



```
#include <stdio.h>

struct aluno{
    int ra;
    char nome[50];
    char endereco[50];
};

int main(){
    struct aluno A;
    ...
    return 0;
}
```

Introdução

- Os registros (ou estruturas) permitem colocar, em uma única entidade, elementos de tipos diferentes.
- Estruturas de dados heterogêneas.
- Os elementos de uma estrutura se chamam campos ou membros.
- Os campos ou membros podem ser de qualquer tipo, incluindo vetores, matrizes e outras estruturas.

Introdução

- Como acontece com qualquer outro tipo de dado, uma variável registro (estrutura) pode ser simples, um vetor ou uma matriz.

Introdução

- Exemplo 2 (dados de um funcionário):

INSC 224881	NOME Fulano de Tal	ENDEREÇO			
		RUA, Nº, APTO São João, 33	CEP 38400-012	CIDADE Uberlândia	UF MG
SITUAÇÃO NORMAL	DOCUMENTOS				
	DOC. MILITAR 3 04 11 024561	TÍTULO 27B MG 66121	CPF 11456510460	CART. PROF. 545 4421321	IDENT. 12426339 MG
CONTA BANCÁRIA 104 0333 001 000129995		ESTADO CIVIL Solteiro	CARGO Professor	SALÁRIO 5.624,00	

Introdução

- Exemplo 2 (dados de um funcionário):

INSC 224881	NOME Fulano de Tal	ENDEREÇO			
		RUA, Nº, APTO São João, 33	CEP 38400-012	CIDADE Uberlândia	UF MG
SITUAÇÃO NORMAL	DOCUMENTOS				
	DOC. MILITAR 3 04 11024561	TÍTULO 27B MG 66121	CPF 11456510460	CART. PROF. 545 4421321	IDENT. 12426339 MG
CONTA BANCÁRIA 104 0333 001 000129995		ESTADO CIVIL Solteiro	CARGO Professor	SALÁRIO 5.624,00	

INSC 656211	NOME Sicrano Beltrano	ENDEREÇO			
		RUA, Nº, APTO Liberdade, 442	CEP 38400-042	CIDADE Uberlândia	UF MG
SITUAÇÃO NORMAL	DOCUMENTOS				
	DOC. MILITAR 1 07 12 114211	TÍTULO 23A MG 44121	CPF 66467810110	CART. PROF. 521 9942143	IDENT. 33123447 MG
CONTA BANCÁRIA 341 8336 001 00014510		ESTADO CIVIL Solteiro	CARGO Operador	SALÁRIO 7.120,00	

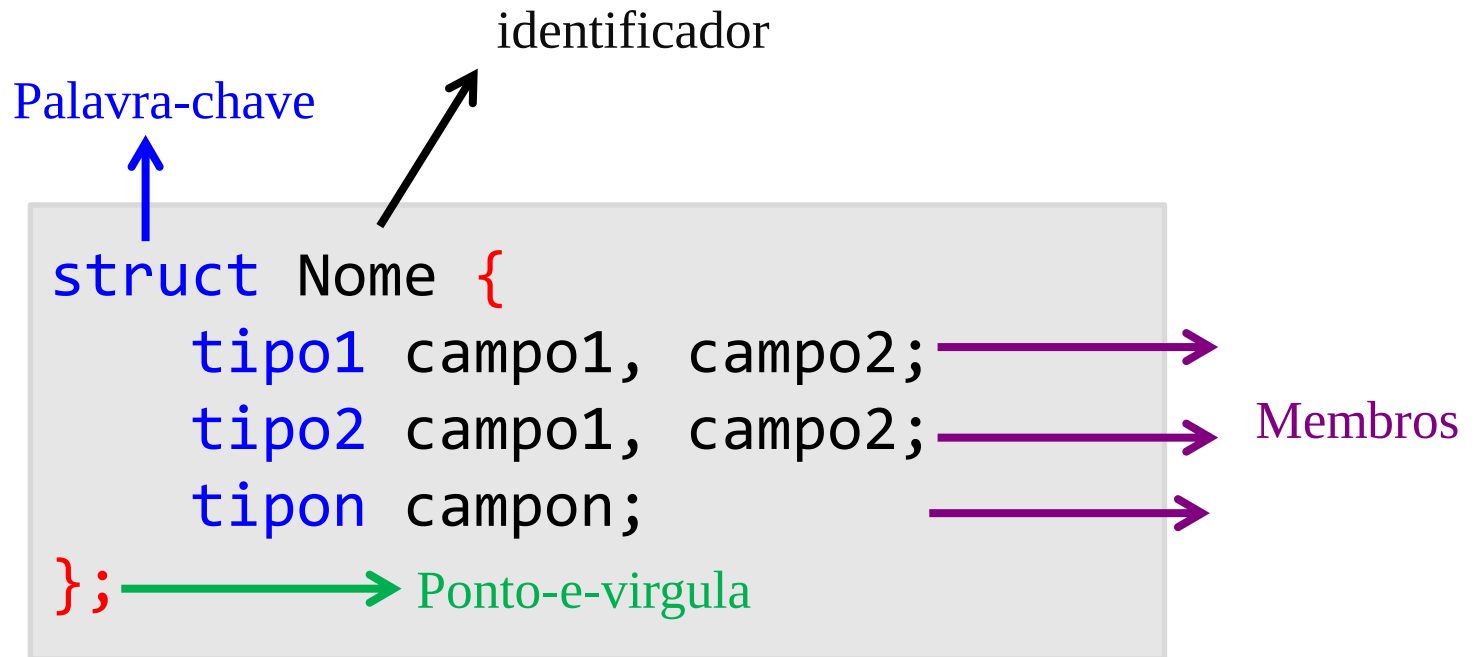
Introdução

- Exemplo 2 (dados de um funcionário):
 - Cada conjunto de informações do funcionário pode ser referenciado por um mesmo nome (por exemplo, ficha)
 - A notação `ficha.nome` é uma referência ao campo `nome` do registro `ficha` cujo conteúdo é “Fulano de Tal”.
 - O objetivo do **conceito de registro** é facilitar o agrupamento de variáveis que não são do mesmo tipo, mas que estão logicamente relacionadas

Introdução

- Na linguagem C, as **estruturas** são o equivalente a registros
- Uma **declaração de estrutura** forma uma entidade que pode ser utilizada para criar variáveis de estrutura
- As variáveis que compreendem a estrutura são chamadas de elementos da estrutura ou membros da estrutura

Sintaxe de declaração



A **definição** de uma estrutura **não cria nenhuma variável**, somente informa ao compilador as características de um novo tipo de dado.

Sintaxe da declaração

```
struct identificador {
    declaração-das-variáveis-
    membros;
} lista-de-identificadores;
```

<i>identificador</i>	Nome escolhido para a estrutura
<i>lista-de-identificadores</i>	Nomes escolhidos para as variáveis de estrutura, separados por vírgula

Exemplo:

```
– struct ficha {
    char nome[31];
    char endereco[41];
    char cep[10];
    char cidade[21];
    char uf[3];
};
```

// Nenhuma variável foi declarada

```
– struct ficha cliente1, cliente2,
    cliente3;

– struct {
    char nome[31];
    char endereco[41];
    char cep[10];
    char cidade[21];
    char uf[3];
} cliente;
```

Acesso aos membros de uma estrutura

- Para acessar os membros de uma instância da estrutura, utiliza-se o operador ponto (.) fazendo **variavel.membro.**

```
#include <stdio.h>
#include <string.h>

struct aluno{
    int ra;
    char nome[50];
    char endereco[50];
};
```

```
int main(){
    struct aluno A;
    A.ra = 1;
    strcpy(A.nome, "João Carlos");
    strcpy(A.endereco, "Rua um");
```

```
printf("\nDados do Aluno:\n");
printf("RA: %d\n", A.ra);
printf("Nome: %s\n", A.nome);
printf("Endereço: %s\n", A.endereco);
return 0;
}
```

Exemplo 1

```
#include <stdio.h>
#include <string.h>

struct aluno{
    int ra;
    char nome[50];
    char endereco[50];
};

int main(){
    //Declaração de variáveis
    struct aluno A;

    //Entrada de Dados
    printf("Digite o RA do aluno: ");
    scanf("%d",&A.ra);
    printf("Digite o nome do aluno: ");
    scanf("%s",A.nome);
    printf("Digite o endereço do aluno: ");
    scanf("%s",A.endereco);

    //Saída de dados
    printf("\nDados do Aluno:\n");
    printf("RA: %d\n",A.ra);
    printf("Nome: %s\n",A.nome);
    printf("Endereço: %s\n", A.endereco);
    return 0;
}
```

Veremos na aula de Strings em C, como manipular de forma mais eficiente a entrada de dados com strings.

Declaração de variáveis

- Uma vez que as estruturas são definidas é possível declarar variáveis deste novo tipo (também é possível criar vetores deste novo tipo).

```
#include <stdio.h>
#include <string.h>

struct aluno{
    int ra;
    char nome[50];
    char endereco[50];
};

int main(){
    //Declaração de variáveis
    struct aluno A1, A2;
    struct aluno A[100];
    //...
    return 0;
}
```

Acesso aos membros de uma estrutura

```
#include <stdio.h>
#include <string.h>

struct aluno{
    int ra;
    char nome[50];
    char endereco[50];
};

int main(){
    //Declaração de variáveis
    struct aluno A[100];

    //Preenchendo os dados
    A[0].ra = 1;
    strcpy(A[0].nome, "João Carlos");
    strcpy(A[0].endereco, "Rua um");

    A[1].ra = 2;
    strcpy(A[1].nome, "Pedro da Silva");
    strcpy(A[1].endereco, "Rua dois");

    return 0;
}
```

Exemplo de representação no vetor

```
struct aluno ALUNOS[3]
```

ALUNOS[0]	ALUNOS[1]	ALUNOS[2]
RA NOME END . . .	RA NOME END . . .	RA NOME END . . .

Exemplo 2

```
#include <stdio.h>
#include <string.h>

struct aluno{
    int ra;
    char nome[50];
    char endereco[50];
};

#define tam 5

int main(){
    //Declaração de variáveis
    struct aluno A[tam];
    int i;

    //Preenchendo os dados

    for(i = 0; i < tam; i++){
        printf("Informe os dados do Aluno %d:\n",i + 1);
        printf("RA: ");
        scanf("%d",&A[i].ra);
        printf("Nome: ");
        scanf("%s", A[i].nome);
        printf("Endereço: ");
        scanf("%s", A[i].endereco);
    }
```

Exemplo 2

```
//Exibindo os dados dos alunos
for(i = 0; i < tam; i++){
    printf("\nDados do aluno %d\n\n",i + 1);
    printf("RA: %d\n",A[i].ra);
    printf("Nome: %s\n",A[i].nome);
    printf("Endereço: %s\n",A[i].endereco);
    printf("-----\n");
}

return 0;
}
```

Atribuição de estruturas

- Os elementos da estrutura do lado esquerdo da atribuição recebem os valores dos elementos correspondentes da estrutura do lado direito.

- Exemplo:

```
struct frac {           // Fração
    int num;            // Numerador
    int den;            // Denominador
} f1, f2;               // Duas variáveis do tipo frac

f1.num = 3;
f1.den = 4;
f2 = f1;                // Atribui uma estrutura à outra

printf("%d/%d\n", f2.num, f2.den);
```

Definição de tipo

- O comando `typedef` pode ser utilizado para nomear tipos de estrutura.
 - Não cria um novo tipo.
 - Permite que um determinado tipo possa ser denominado de forma diferente.
- Evita a necessidade de se usar a palavra **struct** durante a declaração de variáveis.
- Pode ser feito durante ou após a declaração da estrutura.

Exemplo

- Representar os principais dados de um aluno:

```
struct aluno{  
    int ra;  
    char nome[60];  
    char endereco[50];  
};  
  
typedef struct aluno Aluno;
```

Exemplo

- Representar os principais dados de um indivíduo:

```
struct Pessoa{
    int idade;
    char sexo, estadoCivil;
    char nome[60];
    float salario;
    int dia, mes, ano;
};
```

```
typedef struct Pessoa Pessoa;
```

Exemplo

- Representar uma data e hora exata:

```
struct DataHora{  
    int dia, mes, ano;  
    int hora, min, seg;  
};  
  
typedef struct DataHora timestamp;
```

Exemplo

- Representar os principais dados de um indivíduo:

```
typedef struct Pessoa{
    int idade;
    char sexo, estadoCivil;
    char nome[60];
    float salario;
    int dia, mes, ano;
}Pessoa;
```

Exemplo

- Representar os principais dados de um indivíduo:

```
typedef struct{  
    int idade;  
    char sexo, estadoCivil;  
    char nome[60];  
    float salario;  
    int dia, mes, ano;  
}Pessoa;
```

Exemplos de uso

```
Pessoa p1, p2, p3;
```

```
Pessoa funcionarios[100];
```

```
timestamp dtinicio, dtfim;
```

```
Aluno a1, a2;
```

Exercícios

Conta Bancária	
Número	2245
Tipo	001
Titular	José da Silva
Saldo	565,00
Limite	1.000,00

Exercícios

Conta Bancária	
Número	2245
Tipo	001
Titular	José da Silva
Saldo	565,00
Limite	1.000,00

```
struct conta {
    int numero;
    char tipo[4];
    char titular[35];
    double saldo;
    double limite;
};
```

// Conta bancária
// Número da conta
// Tipo (001, 003, 013,...)
// Titular da conta
// Saldo
// Limite

```
struct conta cc;
```

```
cc.numero = 2245;
strcpy(cc.tipo, "001");
strcpy(cc.titular, "José da Silva");
cc.saldo = 565.00;
cc.limite = 1000.00;
```

Exercícios

Contas



Conta Bancária

Conta Bancária	
Número	2245
Tipo	001
Titular	José da Silva
Saldo	565,00
Limite	1.000,00

```
struct conta {           // Conta bancária
    int numero;           // Número da conta
    char tipo[4];         // Tipo (001, 003, 013,... )
    char titular[35];     // Titular da conta
    double saldo;         // Saldo
    double limite;        // Limite
};
```

```
struct conta cc[5];
```

```
cc[2].numero = 2245;
strcpy(cc[2].tipo, "001");
strcpy(cc[2].titular, "José da Silva");
cc[2].saldo = 565.00;
cc[2].limite = 1000.00;
```

Exercícios

	CÓDIGO	NOME
1	001000	ALFAIATE
2	001050	ALMOXARIFE
3	002000	ANALISTA
4	002050	ANTROPÓLOGO
5	003000	BOMBEIRO
6	003050	CARPINTEIRO
	.	.
	.	.
	.	.
99	198 050	TORNEIRO
100	199 000	VENDEDOR

```

struct prof {                // Profissão
    char codigo[7];          // Código da profissão
    char nome[16];           // Nome da profissão
};

struct prof tabela[100];     // Tabela de profissões

```

Estruturas aninhadas

- É possível definir membros da estrutura que são outras estruturas ou vetores.
- Exemplo:

```
typedef struct Funcionario{
    char nome[60];
    float vendas_mes[12];
    int dia_nasc, mes_nasc, ano_nasc;
    int dia_ad, mes_ad, ano_ad;
}Funcionario;
```

Estruturas aninhadas

- É possível definir membros da estrutura que são outras estruturas ou vetores.
- Exemplo:

```
typedef struct Data{
    int dia, mes, ano;
} Data;
```

Estruturas aninhadas

- É possível definir membros da estrutura que são outras estruturas ou vetores.
- Exemplo:

```
typedef struct Funcionario{
    char nome[60];
    float vendas_mes[12];
    Data nascimento, admissao;
}Funcionario;
```

Exemplos de uso

```

Funcionario novo;
Funcionario lista[100];

...
novo.admissao.dia = 22;
novo.admissao.mes = 8;
novo.admissao.ano = 2022;
...

printf("Funcionario %s\n", lista[0].nome);
printf("Vendas em Jan: R$ %f\n", lista[0].vendas_mes[0]);
printf("Ano de nasc: %d\n", lista[0].nascimento.ano);

```

Exercícios

INSC	NOME	ENDEREÇO			
224881	Fulano de Tal	RUA, Nº, APTO	CEP	CIDADE	UF
		São João, 33	38400-012	Uberlândia	MG

SITUAÇÃO	DOCUMENTOS				
NORMAL	DOC. MILITAR	TÍTULO	CPF	CART. PROF.	IDENT.
	3 04 11 024561	27B MG 66121	11456510460	545 4421321	12426339 MG

CONTA BANCÁRIA	ESTADO CIVIL	CARGO	SALÁRIO
104 0333 001 000129995	Solteiro	Professor	5.624,00

Exercícios

ENDEREÇO			
RUA, N°, APTO	CEP	CIDADE	UF
São João, 33	38400-012	Uberlândia	MG

```

struct endereco {
    char logradouro[41];    // Rua, Av. etc.
    char numero[7];        // Número
    char complemento[36];  // Ap., Bloco etc.
    char cep[10];          // 99999-999
    char cidade[41];       // Cidade
    char uf[3];            // AA
};
    
```

Exercícios

DOCUMENTOS				
DOC. MILITAR	TÍTULO	CPF	CART. PROF.	IDENT.
3 04 11 024561	27B MG 66121	11456510460	545 4421321	12426339 MG

```

struct documentos {
    char doc_mil[13];    // Registro de Alistamento
    char titulo[13];    // 004356870906
    char cpf[12];       // 999.999.999-99
    char cart_prof[15]; // Número, série e UF
    char ident[12];     // Padrão
};

```

Variáveis de estrutura

INSC 224881	NOME Fulano de Tal	RUA, Nº, APTO São João, 33	CEP 38400-012	CIDADE Uberlândia	UF MG
SITUAÇÃO NORMAL	DOC. MILITAR 3 04 11 024561	TÍTULO 27B MG 66121	CPF 11456510460	CART. PROF. 545 4421321	IDENT. 12426339 MG
CONTA BANCÁRIA 104 0333 001 000129995	ESTADO CIVIL Solteiro	CARGO Professor	SALÁRIO 5.624,00		

```
struct endereco {
    char logradouro[41];    // Rua, Av. etc.
    char numero[7];        // Número
    char complemento[36];  // Ap., Bloco etc.
    char cep[10];          // 99999-999
    char cidade[41];       // Cidade
    char uf[3];            // AA
};
```

```
struct documentos {
    char doc_mil[13];      // Registro de Alistamento
    char titulo[13];      // 004356870906
    char cpf[12];         // 999.999.999-99
    char cart_prof[15];   // Número, série e UF
    char ident[12];       // Padrão
};
```

```
struct ficha {
    int insc;              // Ficha de inscrição
    char nome[41];         // Inscrição
    struct endereco end;   // Nome
    char situacao[16];     // Endereço
    struct documentos doc; // Situação
    char conta_b[23];      // Documentos
    char est_civil[11];    // Conta bancária
    char cargo[16];        // Estado civil
    float salario;         // Cargo
                          // Salário
};
```

Exercícios

INSC	NOME	ENDEREÇO			
224881	Fulano de Tal	RUA, Nº, APTO	CEP	CIDADE	UF
		São João, 33	38400-012	Uberlândia	MG

SITUAÇÃO	DOCUMENTOS				
NORMAL	DOC. MILITAR	TÍTULO	CPF	CART. PROF.	IDENT.
	3 04 11 024561	27B MG 66121	11456510460	545 4421321	12426339 MG

CONTA BANCÁRIA	ESTADO CIVIL	CARGO	SALÁRIO
104 0333 001 000129995	Solteiro	Professor	5.624,00

Inscrição 0

INSC	NOME	ENDEREÇO			
656211	Sicrano Beltrano	RUA, Nº, APTO	CEP	CIDADE	UF
		Liberdade, 442	38400-042	Uberlândia	MG

SITUAÇÃO	DOCUMENTOS				
NORMAL	DOC. MILITAR	TÍTULO	CPF	CART. PROF.	IDENT.
	1 07 12 114211	23A MG 44121	66467810110	521 9942143	33123447 MG

CONTA BANCÁRIA	ESTADO CIVIL	CARGO	SALÁRIO
341 8336 001 00014510	Solteiro	Operador	7.120,00

Inscrição 1

```

struct endereco {
    char logradouro[41];    // Rua, Av. etc.
    char numero[7];        // Número
    char complemento[36];  // Ap., Bloco etc.
    char cep[10];          // 99999-999
    char cidade[41];       // Cidade
    char uf[3];            // AA
};

struct documentos {
    char doc_mil[13];      // Registro de Alistamento
    char titulo[13];       // 004356870906
    char cpf[12];          // 999.999.999-99
    char cart_prof[15];    // Número, série e UF
    char ident[12];        // Padrão
};

struct ficha {
    // Ficha de inscrição
    int insc;              // Inscrição
    char nome[41];         // Nome
    struct endereco end;   // Endereço
    char situacao[16];     // Situação
    struct documentos doc; // Documentos
    char conta_b[23];      // Conta bancária
    char est_civil[11];    // Estado civil
    char cargo[16];        // Cargo
    float salario;         // Salário
};

```

```
struct ficha inscricao[50]; // Fichas de inscrição
```

```
strcpy(inscricao[0].end.logradouro, "Av. Alberto Carazzai");  
strcpy(inscricao[0].end.numero, "1640");  
strcpy(inscricao[0].end.complemento, "Centro");  
strcpy(inscricao[0].end.cep, "86300-000");  
strcpy(inscricao[0].end.cidade, "Cornélio Procópio");  
strcpy(inscricao[0].end.uf, "PR");
```

```
// Saída formatada para o endereço  
printf("%s, %s - %s\n%s - %s\n%s\n",  
       inscricao[0].end.logradouro,  
       inscricao[0].end.numero,  
       inscricao[0].end.complemento,  
       inscricao[0].end.cidade,  
       inscricao[0].end.uf,  
       inscricao[0].end.cep);
```

Av. Alberto Carazzai, 1640 - Centro
Cornélio Procópio - PR
86300-000

Exercício 2

- Crie um programa em linguagem C para armazenar os dados de um aluno. A estrutura deve armazenar o registro acadêmico do aluno, disciplina matriculada, as notas das quatro avaliações cobradas na disciplina e a média final do aluno. O usuário deverá cadastrar o aluno e exibir suas informações.
- Primeiro, faça o exercício pensando no cadastro de um único aluno e, depois, pense em como poderíamos generalizar o algoritmo para cadastrar e exibir 10 alunos.